

Pholama: A Seven-Stage Pipeline for Compressing and Specializing a 3B-Parameter Large Language Model Toward Ultra-Low-Resource On-Device Deployment

Nawat Pimolsathien
Independent Research Project
nawatpim@gmail.com

June 20, 2026

Abstract

Large language models (LLMs) deliver strong natural-language understanding but their memory and compute footprints preclude deployment on small, resource-constrained devices. This report presents **Pholama**, an end-to-end seven-stage pipeline that compresses and specializes *Typhoon 2 3B*—a Llama 3.2-based Thai-English instruction model of 3.213 billion parameters—into a domain expert for question answering about the Office of the Registrar, Chulalongkorn University. The pipeline combines (i) *vocabulary trimming* of unused-language tokens, (ii) *structured layer pruning* guided by domain calibration data, (iii) *QLoRA recovery fine-tuning*, (iv) *post-training quantization* to the GGUF Q4_K_M format, and (v) *retrieval-augmented generation (RAG)* at deployment. The resulting model is **20.5% smaller** in parameters (3.21 B $\rightarrow$ 2.55 B), occupies a **1.53 GB** quantized deployment file (a 76.2% reduction versus the original bf16 weights), runs **2.2 $\times$ faster** on a bare CPU (23.5 tok/s vs. 10.6 tok/s on GPU), and *improves* in-domain answer quality by 35.7% relative (35.0% $\rightarrow$ 47.5% under RAG). Out-of-domain ability (code/mathematics) degrades as an *intended* consequence of specialization. The entire pipeline was executed on a single consumer-grade RTX 3050 Laptop GPU with 4 GB of VRAM, demonstrating that aggressive, research-grade LLM compression is achievable without cloud infrastructure. We position this work as a step toward the longer-term objective of running specialized language models on extremely small devices such as wearables or pocket calculators.

Contents

1	Introduction	1
1.1	Background and Motivation	1
1.2	Objectives	2
1.3	Contributions	2
2	Methodology	2
2.1	Base Model Specification	2
2.2	System Architecture and Pipeline Overview	3
2.3	Phase 1 — Environment and Baseline	4
2.4	Phase 2 — Domain Dataset Construction	4
2.5	Phase 3 — Vocabulary Trimming	4
2.6	Phase 4 — Structured Pruning	4
2.7	Phase 5 — QLoRA Recovery Fine-Tuning	5
2.8	Phase 6 — Quantization	5

2.9	Phase 7 — Deployment and Evaluation	5
2.10	The Three-Baseline Framework	5
2.11	Reproducibility and Hardware Notes	6
3	Results and Discussion	6
3.1	Axis 1 — Size	6
3.2	Axis 2 — Efficiency (Resources and Speed)	6
3.3	Axis 3 — Quality	7
3.4	Discussion	7
4	Conclusion and Future Work	8
4.1	Conclusion	8
4.2	Future Work	8
	Acknowledgments	8
	References	8

1 Introduction

1.1 Background and Motivation

The capabilities of large language models scale with parameter count, but so do their storage, memory-bandwidth, and energy requirements. A typical 3B model stored in 16-bit precision occupies roughly 6.4 GB and requires a GPU for interactive inference, placing it beyond the reach of phones, microcontrollers, embedded systems, and other ultra-low-resource hardware. Yet many practical applications—an institutional FAQ assistant, an offline help desk, a single-domain chatbot—do not require the model’s full general intelligence. They require fluent comprehension and reasoning *within a narrow domain*, with factual grounding supplied externally.

This observation motivates a *compress-and-specialize* strategy: rather than preserving a model’s general competence, we deliberately discard the parts that serve out-of-domain tasks (e.g. multilingual coverage, code, and mathematics) and reinvest the saved capacity into a smaller, faster, domain-focused model. The chosen domain is Thai–English question answering for the Office of the Registrar, Chulalongkorn University (`reg.chula.ac.th`), a setting with stable terminology and a well-bounded knowledge base.

1.2 Objectives

The project pursues the following concrete objectives:

- (O1) **Reduce model size.** Shrink the 3.21 B base model toward the 1.8 B–2.5 B range while keeping the deployment file within 1.5 GB to 2 GB of RAM after quantization.
- (O2) **Enable on-device / CPU-only inference.** Produce an artifact that runs at interactive speed without a discrete GPU.
- (O3) **Preserve (and ideally improve) in-domain quality** through recovery fine-tuning and retrieval augmentation, while accepting out-of-domain regression as evidence of successful specialization.
- (O4) **Maintain scientific rigor** via a three-baseline comparison framework and reproducible, single-machine execution.

The stated *long-term* aspiration of the Pholama project is to push LLM optimization far enough that a competent, single-purpose language model can run on extremely small devices—a smartwatch or even a calculator-class microcontroller. The present report documents the first milestone toward that vision: a fully functioning, CPU-deployable specialized model derived end-to-end on commodity hardware.

1.3 Contributions

- A reproducible seven-stage compression-and-specialization pipeline, each stage executed and validated on a single 4 GB-VRAM laptop GPU.
- An empirical characterization of vocabulary trimming on an *English-dominant* tokenizer, correcting the common assumption that “dropping dead languages” alone yields large savings.
- A pruning sensitivity analysis showing that the textbook ShortGPT 28→20 configuration collapses the model, whereas a knee-point of 28→24 remains recoverable.
- A full original-versus-specialized evaluation across three axes (size, efficiency, quality) with a deployable RAG demonstrator.

2 Methodology

2.1 Base Model Specification

The base model is Typhoon 2 3B, an instruction-tuned Thai–English model built on the Llama 3.2 3B architecture. Its verified configuration (Table 1) was confirmed against the published `config.json` prior to any modification.

Table 1: Verified architecture of the base model (Typhoon 2 3B).

Hyperparameter	Value
Hidden size d	3072
Number of transformer layers	28
Attention heads	24
Key/value heads (GQA)	8
Head dimension	128
FFN intermediate size	8192
Vocabulary size	128,256
Tied input/output embeddings	true
Total parameters	3,212,749,824

A parameter-budget analysis (Table 2) reveals two facts that shape the entire compression strategy: (1) the tied embedding accounts for only $\sim 12\%$ of the parameters—so vocabulary trimming helps but is not the dominant lever; and (2) within each transformer layer, the feed-forward network (FFN) dominates at $\sim 75\%$, making it the prime target for width pruning.

Table 2: Where the parameters live in the base model.

Component	Parameters	Share
Embedding (tied input + output)	394,000,000	12.3%
28 transformer layers	2,819,000,000	87.7%
Attention (per layer)	25,200,000	25% of layer
FFN / MLP (per layer)	75,500,000	75% of layer

2.2 System Architecture and Pipeline Overview

The Pholama pipeline (Figure 1) transforms the base model through seven sequential stages. Stages 3–6 monotonically reduce model size; stage 5 restores quality; stage 7 supplies external factual grounding so that volatile facts (deadlines, regulations) need not be encoded in the weights.

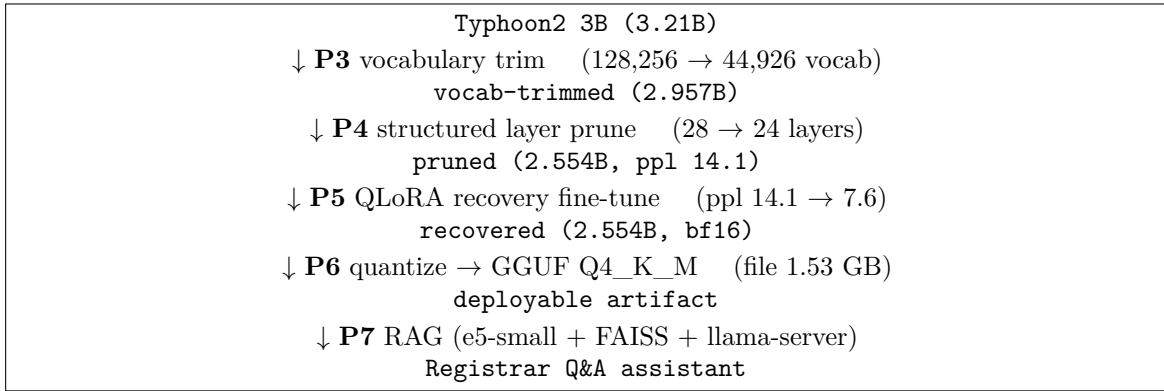


Figure 1: The seven-stage Pholama compression-and-specialization pipeline. Placeholder for a rendered diagram (figures/pipeline.pdf).

2.3 Phase 1 — Environment and Baseline

Tooling comprises PyTorch (CUDA 12.4), Hugging Face `transformers/peft/bitsandbytes`, FAISS, and `llama.cpp`. The base model was loaded in 4-bit and profiled to establish baseline B1 (latency, VRAM, and parameter count). Parameter counts are computed from the configuration rather than `numel()`, because 4-bit packing halves the reported element count.

2.4 Phase 2 — Domain Dataset Construction

A web crawl of `reg.chula.ac.th` (52 HTML pages and ~ 30 PDFs) produced 361 text chunks. A large teacher model (Gemini 2.5 Flash) generated 4,198 synthetic Thai–English question–answer pairs grounded in those chunks, spanning formal/informal registers and Thai/English/code-switched phrasing. Heuristic filtering (self-reference, no-information, length, and near-duplicate removal) reduced the set to 4,016 clean pairs, split 80/10/10 into **3,212 / 401 / 403** train/validation/test items. A leak assertion guarantees that no test item appears in calibration or training data. The same corpus is reused for vocabulary analysis (P3), pruning calibration (P4), recovery training (P5), and retrieval (P7).

2.5 Phase 3 — Vocabulary Trimming

Following Ushio et al. [1], dead tokens are removed from the tied `embed_tokens/lm_head` matrix. Running the tokenizer over the corpus plus train and validation sets (never the test set) yielded

5,388 distinct tokens. A key empirical finding contradicts the original plan’s assumption: the Llama 3.2 tokenizer is *English-dominant* (ASCII/English $\approx 97,718$ tokens; Thai only 2,462; genuinely dead languages—CJK, Korean, Arabic, Cyrillic—only $\approx 24,044$). Dropping dead languages *alone* removes just 18.8% of the vocabulary. To reach the target $\sim 40k$, an *aggressive* mode additionally drops rarely-used English/code tokens by tiktoken frequency rank, retaining the byte alphabet (256), special tokens, all Thai, common Latin-1, punctuation, corpus tokens, and the lowest-id (most frequent) English tokens, plus the transitive closure of BPE merges. The result is a vocabulary of **44,926** tokens (a 65.1% reduction) and a model of 2.9568 **B**. Any text remains encodable via byte fallback.

2.6 Phase 4 — Structured Pruning

Layer pruning. Layer importance is estimated with the ShortGPT Block-Influence criterion—the angular (cosine) distance between a layer’s input and output hidden states under domain calibration data:

$$\text{BI}(\ell) = 1 - \mathbb{E}_x \left[\frac{\mathbf{h}_{\text{in}}^{(\ell)}(x) \cdot \mathbf{h}_{\text{out}}^{(\ell)}(x)}{\|\mathbf{h}_{\text{in}}^{(\ell)}(x)\| \|\mathbf{h}_{\text{out}}^{(\ell)}(x)\|} \right]. \quad (1)$$

Layers with the smallest BI contribute least and are removed; the remaining layers are re-indexed (`layer_idx` reassignment is required for correct rotary-position behavior). A sweep over the number of removed layers (Table 3) shows a sharp “knee”: removing 4 layers (28 $\rightarrow$ 24) yields a recoverable perplexity of 13.2, whereas removing 6 or 8 collapses the model. The plan’s textbook 28 $\rightarrow$ 20 target is therefore *infeasible* for this model.

Width pruning (aggressive variant). For an even smaller variant, the FFN intermediate dimension is reduced 8192 $\rightarrow$ 6144 (−25%) using Wanda-style activation importance, producing a 2.1011 **B** model at the cost of higher perplexity (18.6).

Table 3: Layer-pruning sensitivity sweep (validation perplexity, pre-recovery).

Layers removed	Resulting depth	Val. perplexity
0 (vocab-trimmed base)	28	5.77
2	26	7.2
4	24	13.2 (chosen knee)
6	22	131
8	20	237 (output degenerate)

2.7 Phase 5 — QLoRA Recovery Fine-Tuning

The pruned model is fine-tuned with QLoRA: a 4-bit (nf4) base with LoRA adapters of rank 16 and $\alpha = 32$ applied to the `q,k,v,o,gate,up,down` projections. Training used sequence length 512, batch size 1 with gradient accumulation 16 (effective batch 16), two epochs (402 steps), a cosine schedule with peak learning rate 2×10^{-4} , and *prompt masking* so that loss is computed only over answer tokens. Critically, the adapter is **merged onto a bf16 base**, not the 4-bit base: merging onto the quantized base corrupts perplexity (9 $\rightarrow$ 17) and prevents GGUF conversion. Recovery reduced perplexity from **14.14 to 7.59** (−46%). Peak training memory was only 2.4 GB, well below the plan’s 6 GB to 12 GB estimate, owing to the small adapter rank, gradient checkpointing, and a plain AdamW optimizer.

2.8 Phase 6 — Quantization

The recovered bf16 model is converted to GGUF via `convert_hf_to_gguf.py` and quantized with `llama-quantize` to multiple levels (Q8_0, Q5_K_M, and the deployment target **Q4_K_M**). The trimmed tokenizer required a one-line patch (`res="llama-bpe"`) to satisfy the converter’s pre-tokenizer check. The Q4_K_M artifact is **1.53 GB**, runs at 23.5 tok/s on CPU, and consumes ≈ 1.9 GB of RAM in total—within the target envelope.

2.9 Phase 7 — Deployment and Evaluation

Deployment (RAG). The 361 corpus chunks are embedded with `intfloat/multilingual-e5-small` (384-dim) and indexed with FAISS. At query time the top-4 retrieved chunks are injected into the context of a `llama-server` instance serving the Q4_K_M model (OpenAI-compatible API). This decouples volatile facts from the weights: documents can be updated without retraining.

Evaluation. Quality is measured on a 40-item subset of the test set, with identical RAG for all conditions, judged by an LLM (Gemini) using $\text{score} = \text{correct} + 0.5 \times \text{partial}$. Efficiency (size, RAM, latency) and out-of-domain regression (math/code probes) complete the three-axis evaluation.

2.10 The Three-Baseline Framework

Per the project’s rigor requirement, all results are framed against three baselines (Table 4). The present report fully establishes the **B1**↔**B3** comparison; the B2 ablation and official `lm-eval-harness` runs are identified as remaining work.

Table 4: The three mandatory baselines.

ID	Configuration	Purpose	Status
B1	Original Typhoon 2 3B	Upper-bound general model	✓ measured
B2	Original + QLoRA + quantize (no prune)	Isolate the prune effect	future work
B3	Full pipeline (trim + prune + recover + quant.)	The proposed model	✓ measured

2.11 Reproducibility and Hardware Notes

All stages ran on Windows 11 with an RTX 3050 Laptop GPU (4 GB VRAM); bf16 stages were made feasible by CUDA system-memory fallback (spilling to host RAM, $\sim 9\times$ faster than CPU). Several platform-specific pitfalls were encountered and worked around: importing `transformers.Trainer/datasets` triggers a PyArrow segfault (addressed by a hand-written training loop); `sentence_transformers` likewise segfaults (addressed by calling `AutoModel` directly); and the GGUF tooling’s pip dependencies can silently downgrade PyTorch to a CPU build.

3 Results and Discussion

3.1 Axis 1 — Size

Table 5 summarizes the size reduction. The 20.5% parameter reduction decomposes into vocabulary trimming (-256M , via dropped unused-language tokens) and layer pruning ($28 \rightarrow 24$,

−4 layers); the remaining file-size reduction comes from Q4_K_M quantization. Relative to the original bf16 weights, the deployable artifact is **76.2% smaller**.

Table 5: Axis 1 — size comparison (original B1 vs. specialized B3).

Metric	Original (B1)	New (B3)	Change
Total parameters	3,212,749,824	2,554,100,000	−20.5%
Vocabulary	128,256	44,926	−65.0%
Transformer layers	28	24	−14.3%
BPE merges	280,147	103,362	−63.1%
Weights (bf16)	6.43 GB	5.11 GB	−20.5%
Deploy weights (quant.)	1.77 GB (4-bit)	1.53 GB (Q4_K_M)	−13.6%
Deploy vs. original bf16	6.43 GB	1.53 GB	− 76.2%

3.2 Axis 2 — Efficiency (Resources and Speed)

Table 6 reports the efficiency results. The headline finding is that the specialized model on a *bare CPU* (23.5 tok/s) is faster than the original on a *GPU* (10.6 tok/s), while requiring no discrete accelerator and ∼15.6% less runtime memory.

Table 6: Axis 2 — efficiency comparison.

Metric	Original (B1)	New (B3)	Change
Runtime RAM	2.25 GB (VRAM)	∼1.9 GB (RAM)	−15.6%
Device required	GPU	CPU is enough	—
Generation speed	10.6 tok/s (GPU)	23.5 tok/s (CPU)	+122%
Prompt speed	—	122 tok/s (CPU)	—
Time / answer (∼60 tok)	∼6 s	∼3 s	−50%

3.3 Axis 3 — Quality

In-domain (improved). Table 7 shows that the specialized model *outperforms* the original on the domain test set, both with and without RAG. Recovery fine-tuning taught the model the domain’s answer style and specific facts (e.g. Thai request-form codes such as “**Jor.Tor.**”, submission channels, and schedules), whereas the original answers generically. RAG benefits both models by supplying ground-truth facts.

Table 7: Axis 3 — in-domain quality (40-item test subset, LLM judge).

Condition	Original (B1)	New (B3)	Change (rel.)
+ RAG	35.0 %	47.5 %	+35.7%
No RAG	16.2 %	41.2 %	+154%

Out-of-domain (intentionally degraded). Probes confirm the loss of general ability (Table 8): arithmetic and code generation fail after specialization. Per the project’s design philosophy, this regression is not a defect but *positive evidence* that the model traded general competence for domain expertise and compactness.

Table 8: Axis 3 — out-of-domain regression probes (new model B3).

Task	B3 answer	Status
$17 \times 23 = ?$	411 (correct: 391)	wrong
Fibonacci function	“uses the recursive method” (no code)	vague
Solve $2x + 5 = 13$	$2x = 8$ (does not finish $x = 4$)	incomplete

3.4 Discussion

Where the savings really come from. The parameter budget (Table 2) correctly predicted that vocabulary trimming would be a minor lever for a 3B model ($\sim 12\%$ of parameters live in the embedding) and that structured pruning of the transformer stack is the dominant contributor. The empirical 20.5% reduction is consistent with this: vocab trim contributes $\sim 8\%$ and layer pruning the remainder.

Plan-versus-reality corrections. Two assumptions in the original plan did not survive contact with the model. First, vocabulary trimming cannot reach $\sim 40k$ by dropping dead languages alone, because the tokenizer is English-dominant; reaching the target requires deliberately discarding rarely-used English/code tokens—which is, fortuitously, aligned with the specialization goal. Second, the canonical ShortGPT 28 $\rightarrow$ 20 configuration is infeasible: removing eight contiguous layers raises perplexity to 237 and produces degenerate output. The recoverable knee lies at 28 $\rightarrow$ 24. These corrections are themselves useful contributions, as both assumptions are common in the compression literature.

Cost-effectiveness. Combining the three axes, the specialized model is one-fifth smaller, roughly twice as fast, GPU-free, and *better* in-domain than the original. For a single-purpose deployment this is a clearly favorable trade, achieved entirely on commodity hardware.

Threats to validity. In-domain quality is measured on 40 of 403 test items with an LLM judge—an indicator rather than a certified benchmark. Out-of-domain regression is, so far, a qualitative probe; official `gsm8k/humaneval` numbers are deferred to GPU compute. Finally, B1 was measured on GPU 4-bit and B3 on CPU Q4, reflecting realistic deployment conditions but differing hardware. The B2 ablation, which isolates the pruning contribution from QLoRA+quantization, remains outstanding.

4 Conclusion and Future Work

4.1 Conclusion

This report presented **Pholama**, a complete seven-stage pipeline that compresses and specializes a 3.21 B Thai–English LLM into a 2.55 B, 1.53 GB, CPU-deployable domain expert for Chula Registrar question answering. The model is 20.5% smaller, 122% faster on CPU than the original on GPU, fits within ~ 1.9 GB of RAM, and answers the target domain 35.7% better (relative) under RAG—while shedding out-of-domain ability by design. Every stage was executed on a single 4 GB-VRAM laptop, demonstrating that the full compress-and-specialize workflow is accessible without cloud resources.

4.2 Future Work

- **Complete the baseline matrix.** Run the B2 ablation (original + QLoRA + quantize, no pruning) to isolate the pruning contribution, and obtain official `lm-eval-harness` (`gsm8k`,

humaneval) numbers across B1/B2/B3 on GPU.

- **Push the aggressive variant.** The 2.1011 B FFN-pruned model is not yet recovered/quantized; with recovery it may offer an even smaller deployment point.
- **Toward truly tiny devices.** The project’s long-term goal is sub-gigabyte, microcontroller-class deployment (smartwatch- or calculator-class hardware). Promising directions include extreme quantization (2–3 bit, e.g. ternary/1.58-bit schemes), KV-cache quantization, knowledge distillation into a smaller student, and speculative or streaming decoding to reduce latency on constrained CPUs.
- **Broaden evaluation.** Expand the human-/judge-scored test set beyond 40 items and add calibration and hallucination metrics under RAG.

Acknowledgments

The author thanks the maintainers of the open-source tools that made this work possible—PyTorch, Hugging Face Transformers/PEFT, `bitsandbytes`, FAISS, and `llama.cpp`—and acknowledges the public information published by the Office of the Registrar, Chulalongkorn University, used as the domain corpus.

References

- [1] A. Ushio, Y. Zhou, and J. Camacho-Collados, “An Efficient Multilingual Language Model Compression through Vocabulary Trimming,” in *Findings of EMNLP*, 2023. arXiv:2305.15020.
- [2] X. Men *et al.*, “ShortGPT: Layers in Large Language Models are More Redundant than You Expect,” arXiv:2403.03853, 2024.
- [3] T. Dettmers, A. Pagnoni, A. Holtzman, and L. Zettlemoyer, “QLoRA: Efficient Finetuning of Quantized LLMs,” in *NeurIPS*, 2023.
- [4] E. J. Hu *et al.*, “LoRA: Low-Rank Adaptation of Large Language Models,” in *ICLR*, 2022.
- [5] M. Sun, Z. Liu, A. Bair, and J. Z. Kolter, “A Simple and Effective Pruning Approach for Large Language Models (Wanda),” in *ICLR*, 2024.
- [6] X. Ma, G. Fang, and X. Wang, “LLM-Pruner: On the Structural Pruning of Large Language Models,” in *NeurIPS*, 2023.
- [7] A. Dubey *et al.*, “The Llama 3 Herd of Models,” arXiv:2407.21783, 2024.
- [8] P. Lewis *et al.*, “Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks,” in *NeurIPS*, 2020. arXiv:2005.11401.
- [9] G. Gerganov *et al.*, “llama.cpp: LLM inference in C/C++,” <https://github.com/ggerganov/llama.cpp>, 2023–.